Relationship Tracker

Full Project — MySQL + Python + Tkinter

#	Section
1	Project Structure
2	Setup & Installation
3	Database Schema (schema.sql)
4	Database Connection (db_connection.py)
5	All Queries — Python (queries.py)
6	MySQL Workbench Queries (workbench_queries.sql)
7	Login Window UI (login_window.py)
8	App Entry Point (main.py)

1. Project Structure

```
relationship_tracker/
    main.py
    requirements.txt
    database/
        schema.sql
        db_connection.py
        queries.py
        workbench_queries.sql
    ui/
        login_window.py
```

2. Setup & Installation

1. Install Python packages

pip install mysql-connector-python bcrypt pillow reportlab

2. Create the database

Open MySQL Workbench and run schema.sql

3. Set credentials

Edit DB_CONFIG in database/db_connection.py

4. Run the app

Right-click main.py in PyCharm → Run 'main'

3. Database Schema — schema.sql

Table: users

id, username, email, password_hash, created_at, last_login

Table: contacts

id, user_id, first_name, last_name, nickname, relationship_type, category, email, phone, company, job_title, birthday, city, country, notes, photo_path, is_favourite, created_at, updated_at

Table: interactions

id, contact_id, user_id, interaction_type, subject, description, sentiment, interaction_date, next_follow_up, created_at

Table: tags

id, user_id, name, color

Table: contact_tags

contact_id, tag_id

Table: reminders

id, user_id, contact_id, title, description, due_date, is_done, priority, created_at

Table: relationship_goals

id, user_id, contact_id, title, description, target_date, is_achieved, created_at

Full SQL

```
CREATE DATABASE IF NOT EXISTS relationship_tracker
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE relationship_tracker;

CREATE TABLE IF NOT EXISTS users (
id INT AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(50) NOT NULL UNIQUE,
email VARCHAR(120) NOT NULL UNIQUE,
password_hash VARCHAR(255) NOT NULL,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
last_login DATETIME
);

CREATE TABLE IF NOT EXISTS contacts (
id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,
first_name VARCHAR(60) NOT NULL,
```

```
last_name VARCHAR(60),
nickname VARCHAR(60),
relationship_type ENUM('friend','family','romantic','colleague',
'client','mentor','acquaintance','other') NOT NULL DEFAULT 'other',
category ENUM('personal','professional','both') NOT NULL DEFAULT 'personal',
email VARCHAR(120),
phone VARCHAR(30),
company VARCHAR(100),
job_title VARCHAR(100),
birthday DATE,
city VARCHAR(80),
country VARCHAR(80),
notes TEXT,
photo_path VARCHAR(255),
is_favourite TINYINT(1) DEFAULT 0,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
INDEX idx_user_id (user_id),
INDEX idx_relationship_type (relationship_type),
INDEX idx_category (category)
);

CREATE TABLE IF NOT EXISTS interactions (
id INT AUTO_INCREMENT PRIMARY KEY,
contact_id INT NOT NULL,
user_id INT NOT NULL,
interaction_type ENUM('call','meeting','message','email',
'coffee','lunch','event','gift','other') NOT NULL,
subject VARCHAR(200),
description TEXT,
sentiment ENUM('positive','neutral','negative') DEFAULT 'neutral',
interaction_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
next_follow_up DATE,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (contact_id) REFERENCES contacts(id) ON DELETE CASCADE,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS tags (
id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,
name VARCHAR(50) NOT NULL,
color VARCHAR(7) DEFAULT '#6C63FF',
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
```

```
UNIQUE KEY uq_user_tag (user_id, name)
);

CREATE TABLE IF NOT EXISTS contact_tags (
contact_id INT NOT NULL,
tag_id INT NOT NULL,
PRIMARY KEY (contact_id, tag_id),
FOREIGN KEY (contact_id) REFERENCES contacts(id) ON DELETE CASCADE,
FOREIGN KEY (tag_id) REFERENCES tags(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS reminders (
id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,
contact_id INT,
title VARCHAR(200) NOT NULL,
description TEXT,
due_date DATETIME NOT NULL,
is_done TINYINT(1) DEFAULT 0,
priority ENUM('low','medium','high') DEFAULT 'medium',
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
FOREIGN KEY (contact_id) REFERENCES contacts(id) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS relationship_goals (
id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,
contact_id INT,
title VARCHAR(200) NOT NULL,
description TEXT,
target_date DATE,
is_achieved TINYINT(1) DEFAULT 0,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
FOREIGN KEY (contact_id) REFERENCES contacts(id) ON DELETE SET NULL
);

INSERT IGNORE INTO users (username, email, password_hash) VALUES
('demo_user','demo@example.com',
'$2b$12$KIXHs3J9v3QwY.u0m0m0oOvXz1sR2tL5bN8cP7dQ6eR1fS4gT3hU2');
```

4. Database Connection — db_connection.py

```
import mysql.connector
from mysql.connector import pooling, Error
import bcrypt
from datetime import datetime

DB_CONFIG = {
"host": "localhost",
"port": 3306,
"user": "root",
"password": "your_password",
"database": "relationship_tracker",
"charset": "utf8mb4",
"collation": "utf8mb4_unicode_ci",
}

_pool = None

def get_pool():
global _pool
if _pool is None:
_pool = pooling.MySQLConnectionPool(
pool_name="rt_pool", pool_size=5, **DB_CONFIG)
return _pool

def get_connection():
return get_pool().get_connection()

def hash_password(plain_password: str) -> str:
salt = bcrypt.gensalt(rounds=12)
return bcrypt.hashpw(plain_password.encode("utf-8"), salt).decode("utf-8")

def verify_password(plain_password: str, hashed: str) -> bool:
return bcrypt.checkpw(plain_password.encode("utf-8"), hashed.encode("utf-8"))

def register_user(username: str, email: str, password: str):
hashed = hash_password(password)
sql = "INSERT INTO users (username, email, password_hash) VALUES (%s, %s, %s)"
try:
conn = get_connection()
cursor = conn.cursor()
cursor.execute(sql, (username, email, hashed))
conn.commit()
user_id = cursor.lastrowid
return True, user_id
except Error as e:
msg = "Username or email already exists." if e.errno == 1062 else str(e)
return False, msg
```

```
finally:
cursor.close(); conn.close()

def login_user(username: str, password: str):
sql = "SELECT id, username, email, password_hash FROM users WHERE username = %s"
try:
conn = get_connection()
cursor = conn.cursor(dictionary=True)
cursor.execute(sql, (username,))
user = cursor.fetchone()
if not user:
return False, "Username not found."
if not verify_password(password, user["password_hash"]):
return False, "Incorrect password."
conn2 = get_connection()
cur2 = conn2.cursor()
cur2.execute("UPDATE users SET last_login = %s WHERE id = %s",
(datetime.now(), user["id"]))
conn2.commit(); cur2.close(); conn2.close()
del user["password_hash"]
return True, user
except Error as e:
return False, str(e)
finally:
cursor.close(); conn.close()

def add_contact(user_id: int, data: dict):
sql = """INSERT INTO contacts
(user_id, first_name, last_name, nickname, relationship_type, category,
email, phone, company, job_title, birthday, city, country, notes)
VALUES (%(user_id)s, %(first_name)s, %(last_name)s, %(nickname)s,
%(relationship_type)s, %(category)s, %(email)s, %(phone)s,
%(company)s, %(job_title)s, %(birthday)s, %(city)s, %(country)s, %(notes)s)"""
data["user_id"] = user_id
try:
conn = get_connection(); cursor = conn.cursor()
cursor.execute(sql, data); conn.commit()
return True, cursor.lastrowid
except Error as e:
return False, str(e)
finally:
cursor.close(); conn.close()

def get_contacts(user_id: int, search: str = "", category: str = "all"):
params = [user_id]; where = ["user_id = %s"]
if search:
```

```
where.append("(first_name LIKE %s OR last_name LIKE %s OR company LIKE %s)")
like = f"%{search}%"; params += [like, like, like]
if category != "all":
where.append("category = %s"); params.append(category)
sql = f"SELECT * FROM contacts WHERE {' AND '.join(where)} ORDER BY first_name ASC"
try:
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute(sql, params); return cursor.fetchall()
except Error as e:
return []
finally:
cursor.close(); conn.close()

def log_interaction(user_id: int, contact_id: int, data: dict):
sql = """INSERT INTO interactions
(contact_id, user_id, interaction_type, subject, description,
sentiment, interaction_date, next_follow_up)
VALUES (%(contact_id)s, %(user_id)s, %(interaction_type)s, %(subject)s,
%(description)s, %(sentiment)s, %(interaction_date)s, %(next_follow_up)s)"""
data["contact_id"] = contact_id; data["user_id"] = user_id
try:
conn = get_connection(); cursor = conn.cursor()
cursor.execute(sql, data); conn.commit()
return True, cursor.lastrowid
except Error as e:
return False, str(e)
finally:
cursor.close(); conn.close()

def get_upcoming_reminders(user_id: int, days_ahead: int = 7):
sql = """SELECT r.id, r.title, r.due_date, r.priority, c.first_name, c.last_name
FROM reminders r LEFT JOIN contacts c ON c.id = r.contact_id
WHERE r.user_id = %s AND r.is_done = 0
AND r.due_date BETWEEN NOW() AND DATE_ADD(NOW(), INTERVAL %s DAY)
ORDER BY r.due_date ASC"""
try:
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute(sql, (user_id, days_ahead)); return cursor.fetchall()
except Error as e:
return []
finally:
cursor.close(); conn.close()

def test_connection():
try:
conn = get_connection(); conn.close()
```

```
print("[DB] Connected to MySQL successfully."); return True
except Error as e:
print(f"[DB] Connection failed: {e}"); return False
```

5. All Queries — Python — queries.py

```
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from database.db_connection import get_connection
from mysql.connector import Error

def get_all_contacts(user_id):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT * FROM contacts WHERE user_id = %s ORDER BY first_name ASC",
(user_id,))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def get_contact_by_id(contact_id):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT * FROM contacts WHERE id = %s", (contact_id,))
result = cursor.fetchone(); cursor.close(); conn.close(); return result

def search_contacts(user_id, keyword):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
like = f"%{keyword}%"
cursor.execute("""SELECT * FROM contacts WHERE user_id = %s
AND (first_name LIKE %s OR last_name LIKE %s OR email LIKE %s
OR phone LIKE %s OR company LIKE %s OR city LIKE %s)
ORDER BY first_name ASC""", (user_id, like, like, like, like, like, like))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def get_contacts_by_category(user_id, category):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT * FROM contacts WHERE user_id = %s AND category = %s ORDER BY
first_name ASC", (user_id, category))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def get_contacts_by_relationship(user_id, relationship_type):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT * FROM contacts WHERE user_id = %s AND relationship_type = %s ORDER BY
first_name ASC", (user_id, relationship_type))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def get_favourite_contacts(user_id):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT * FROM contacts WHERE user_id = %s AND is_favourite = 1 ORDER BY
first_name ASC", (user_id,))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def update_contact(contact_id, data):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("""UPDATE contacts SET first_name=%(first_name)s, last_name=%(last_name)s,
nickname=%(nickname)s, relationship_type=%(relationship_type)s, category=%(category)s,
email=%(email)s, phone=%(phone)s, company=%(company)s, job_title=%(job_title)s,
```

```
birthday=%(birthday)s, city=%(city)s, country=%(country)s, notes=%(notes)s
WHERE id=%(id)s""", {**data, "id": contact_id})
conn.commit(); cursor.close(); conn.close()

def delete_contact(contact_id):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("DELETE FROM contacts WHERE id = %s", (contact_id,))
conn.commit(); cursor.close(); conn.close()

def toggle_favourite(contact_id):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("UPDATE contacts SET is_favourite = NOT is_favourite WHERE id = %s",
(contact_id,))
conn.commit(); cursor.close(); conn.close()

def get_interactions_for_contact(contact_id):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT * FROM interactions WHERE contact_id = %s ORDER BY interaction_date
DESC", (contact_id,))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def get_recent_interactions(user_id, limit=10):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("""SELECT i.*, c.first_name, c.last_name FROM interactions i
JOIN contacts c ON c.id = i.contact_id WHERE i.user_id = %s
ORDER BY i.interaction_date DESC LIMIT %s""", (user_id, limit))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def get_interactions_by_type(user_id, interaction_type):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("""SELECT i.*, c.first_name, c.last_name FROM interactions i
JOIN contacts c ON c.id = i.contact_id
WHERE i.user_id = %s AND i.interaction_type = %s
ORDER BY i.interaction_date DESC""", (user_id, interaction_type))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def get_interactions_by_sentiment(user_id, sentiment):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("""SELECT i.*, c.first_name, c.last_name FROM interactions i
JOIN contacts c ON c.id = i.contact_id
WHERE i.user_id = %s AND i.sentiment = %s
ORDER BY i.interaction_date DESC""", (user_id, sentiment))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def delete_interaction(interaction_id):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("DELETE FROM interactions WHERE id = %s", (interaction_id,))
conn.commit(); cursor.close(); conn.close()

def get_all_reminders(user_id):
```

```
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("""SELECT r.*, c.first_name, c.last_name FROM reminders r
LEFT JOIN contacts c ON c.id = r.contact_id
WHERE r.user_id = %s ORDER BY r.due_date ASC""", (user_id,))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def get_pending_reminders(user_id):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("""SELECT r.*, c.first_name, c.last_name FROM reminders r
LEFT JOIN contacts c ON c.id = r.contact_id
WHERE r.user_id = %s AND r.is_done = 0 ORDER BY r.due_date ASC""", (user_id,))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def get_overdue_reminders(user_id):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("""SELECT r.*, c.first_name, c.last_name FROM reminders r
LEFT JOIN contacts c ON c.id = r.contact_id
WHERE r.user_id = %s AND r.is_done = 0 AND r.due_date < NOW()
ORDER BY r.due_date ASC""", (user_id,))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def add_reminder(user_id, contact_id, title, description, due_date, priority="medium"):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("""INSERT INTO reminders (user_id, contact_id, title, description, due_date,
priority)
VALUES (%s, %s, %s, %s, %s, %s)""", (user_id, contact_id, title, description, due_date,
priority))
conn.commit(); rid = cursor.lastrowid; cursor.close(); conn.close(); return rid

def mark_reminder_done(reminder_id):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("UPDATE reminders SET is_done = 1 WHERE id = %s", (reminder_id,))
conn.commit(); cursor.close(); conn.close()

def delete_reminder(reminder_id):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("DELETE FROM reminders WHERE id = %s", (reminder_id,))
conn.commit(); cursor.close(); conn.close()

def get_all_tags(user_id):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT * FROM tags WHERE user_id = %s ORDER BY name ASC", (user_id,))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def add_tag(user_id, name, color="#6C63FF"):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("INSERT INTO tags (user_id, name, color) VALUES (%s, %s, %s)", (user_id, name,
color))
conn.commit(); tid = cursor.lastrowid; cursor.close(); conn.close(); return tid
```

```
def assign_tag_to_contact(contact_id, tag_id):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("INSERT IGNORE INTO contact_tags (contact_id, tag_id) VALUES (%s, %s)", (contact_id, tag_id))
conn.commit(); cursor.close(); conn.close()

def remove_tag_from_contact(contact_id, tag_id):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("DELETE FROM contact_tags WHERE contact_id = %s AND tag_id = %s", (contact_id, tag_id))
conn.commit(); cursor.close(); conn.close()

def get_tags_for_contact(contact_id):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("""SELECT t.* FROM tags t JOIN contact_tags ct ON ct.tag_id = t.id
WHERE ct.contact_id = %s""", (contact_id,))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def get_contacts_by_tag(tag_id):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("""SELECT c.* FROM contacts c JOIN contact_tags ct ON ct.contact_id = c.id
WHERE ct.tag_id = %s ORDER BY c.first_name ASC""", (tag_id,))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def delete_tag(tag_id):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("DELETE FROM tags WHERE id = %s", (tag_id,))
conn.commit(); cursor.close(); conn.close()

def add_goal(user_id, contact_id, title, description, target_date):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("""INSERT INTO relationship_goals
(user_id, contact_id, title, description, target_date)
VALUES (%s, %s, %s, %s, %s)""", (user_id, contact_id, title, description, target_date))
conn.commit(); gid = cursor.lastrowid; cursor.close(); conn.close(); return gid

def get_goals(user_id):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("""SELECT g.*, c.first_name, c.last_name FROM relationship_goals g
LEFT JOIN contacts c ON c.id = g.contact_id
WHERE g.user_id = %s ORDER BY g.target_date ASC""", (user_id,))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def mark_goal_achieved(goal_id):
conn = get_connection(); cursor = conn.cursor()
cursor.execute("UPDATE relationship_goals SET is_achieved = 1 WHERE id = %s", (goal_id,))
conn.commit(); cursor.close(); conn.close()

def delete_goal(goal_id):
conn = get_connection(); cursor = conn.cursor()
```

```
cursor.execute("DELETE FROM relationship_goals WHERE id = %s", (goal_id,))
conn.commit(); cursor.close(); conn.close()

def get_contacts_with_birthday_this_month(user_id):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("""SELECT * FROM contacts WHERE user_id = %s
AND MONTH(birthday) = MONTH(CURDATE()) ORDER BY DAY(birthday) ASC""", (user_id,))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def get_contacts_not_contacted_since(user_id, days):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("""SELECT c.*, MAX(i.interaction_date) AS last_contact FROM contacts c
LEFT JOIN interactions i ON i.contact_id = c.id WHERE c.user_id = %s
GROUP BY c.id HAVING last_contact IS NULL
OR last_contact < DATE_SUB(NOW(), INTERVAL %s DAY)
ORDER BY last_contact ASC""", (user_id, days))
result = cursor.fetchall(); cursor.close(); conn.close(); return result

def get_dashboard_summary(user_id):
conn = get_connection(); cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT COUNT(*) AS total FROM contacts WHERE user_id = %s", (user_id,))
total_contacts = cursor.fetchone()["total"]
cursor.execute("SELECT COUNT(*) AS total FROM interactions WHERE user_id = %s", (user_id,))
total_interactions = cursor.fetchone()["total"]
cursor.execute("SELECT COUNT(*) AS total FROM reminders WHERE user_id = %s AND is_done=0 AND
due_date>=NOW()", (user_id,))
pending_reminders = cursor.fetchone()["total"]
cursor.execute("SELECT COUNT(*) AS total FROM reminders WHERE user_id = %s AND is_done=0 AND
due_date<NOW()", (user_id,))
overdue_reminders = cursor.fetchone()["total"]
cursor.close(); conn.close()
return {"total_contacts": total_contacts, "total_interactions": total_interactions,
"pending_reminders": pending_reminders, "overdue_reminders": overdue_reminders}
```

6. MySQL Workbench Queries — workbench_queries.sql

Contacts

```
SELECT * FROM contacts WHERE user_id = 1 ORDER BY first_name ASC;

SELECT * FROM contacts WHERE id = 1;

SELECT * FROM contacts WHERE user_id = 1

AND (first_name LIKE '%john%' OR last_name LIKE '%john%'

OR email LIKE '%john%' OR company LIKE '%john%') ORDER BY first_name ASC;

SELECT * FROM contacts WHERE user_id = 1 AND category = 'personal' ORDER BY first_name ASC;

SELECT * FROM contacts WHERE user_id = 1 AND category = 'professional' ORDER BY first_name
ASC;

SELECT * FROM contacts WHERE user_id = 1 AND relationship_type = 'friend' ORDER BY first_name
ASC;

SELECT * FROM contacts WHERE user_id = 1 AND is_favourite = 1 ORDER BY first_name ASC;

UPDATE contacts SET first_name='John', last_name='Doe', relationship_type='friend',

category='personal', email='john@example.com', phone='9876543210'

WHERE id = 1;

DELETE FROM contacts WHERE id = 1;

UPDATE contacts SET is_favourite = NOT is_favourite WHERE id = 1;
```

Interactions

```
SELECT * FROM interactions WHERE contact_id = 1 ORDER BY interaction_date DESC;

SELECT i.*, c.first_name, c.last_name FROM interactions i

JOIN contacts c ON c.id = i.contact_id WHERE i.user_id = 1

ORDER BY i.interaction_date DESC LIMIT 10;

SELECT i.*, c.first_name, c.last_name FROM interactions i

JOIN contacts c ON c.id = i.contact_id

WHERE i.user_id = 1 AND i.interaction_type = 'call' ORDER BY i.interaction_date DESC;

SELECT i.*, c.first_name, c.last_name FROM interactions i

JOIN contacts c ON c.id = i.contact_id

WHERE i.user_id = 1 AND i.sentiment = 'positive' ORDER BY i.interaction_date DESC;

INSERT INTO interactions (contact_id, user_id, interaction_type, subject, description,
sentiment, interaction_date, next_follow_up)

VALUES (1, 1, 'call', 'Project Update', 'Discussed timeline', 'positive', NOW(),
'2025-07-01');

DELETE FROM interactions WHERE id = 1;
```

Reminders

```
SELECT r.*, c.first_name, c.last_name FROM reminders r

LEFT JOIN contacts c ON c.id = r.contact_id WHERE r.user_id = 1 ORDER BY r.due_date ASC;

SELECT r.*, c.first_name, c.last_name FROM reminders r

LEFT JOIN contacts c ON c.id = r.contact_id

WHERE r.user_id = 1 AND r.is_done = 0 ORDER BY r.due_date ASC;
```

```
SELECT r.*, c.first_name, c.last_name FROM reminders r
LEFT JOIN contacts c ON c.id = r.contact_id
WHERE r.user_id = 1 AND r.is_done = 0 AND r.due_date < NOW() ORDER BY r.due_date ASC;
SELECT r.*, c.first_name, c.last_name FROM reminders r
LEFT JOIN contacts c ON c.id = r.contact_id
WHERE r.user_id = 1 AND r.is_done = 0
AND r.due_date BETWEEN NOW() AND DATE_ADD(NOW(), INTERVAL 7 DAY) ORDER BY r.due_date ASC;
INSERT INTO reminders (user_id, contact_id, title, description, due_date, priority)
VALUES (1, 1, 'Follow up call', 'Check project status', '2025-07-05 10:00:00', 'high');
UPDATE reminders SET is_done = 1 WHERE id = 1;
DELETE FROM reminders WHERE id = 1;
```

Tags

```
SELECT * FROM tags WHERE user_id = 1 ORDER BY name ASC;
INSERT INTO tags (user_id, name, color) VALUES (1, 'VIP', '#FFD700');
INSERT IGNORE INTO contact_tags (contact_id, tag_id) VALUES (1, 1);
DELETE FROM contact_tags WHERE contact_id = 1 AND tag_id = 1;
SELECT t.* FROM tags t JOIN contact_tags ct ON ct.tag_id = t.id WHERE ct.contact_id = 1;
SELECT c.* FROM contacts c JOIN contact_tags ct ON ct.contact_id = c.id
WHERE ct.tag_id = 1 ORDER BY c.first_name ASC;
DELETE FROM tags WHERE id = 1;
```

Goals

```
SELECT g.*, c.first_name, c.last_name FROM relationship_goals g
LEFT JOIN contacts c ON c.id = g.contact_id WHERE g.user_id = 1 ORDER BY g.target_date ASC;
INSERT INTO relationship_goals (user_id, contact_id, title, description, target_date)
VALUES (1, 1, 'Monthly catch-up', 'Stay in touch', '2025-12-31');
UPDATE relationship_goals SET is_achieved = 1 WHERE id = 1;
DELETE FROM relationship_goals WHERE id = 1;
```

Smart Queries

```
SELECT * FROM contacts WHERE user_id = 1 AND MONTH(birthday) = MONTH(CURDATE())
ORDER BY DAY(birthday) ASC;

SELECT c.*, MAX(i.interaction_date) AS last_contact FROM contacts c
LEFT JOIN interactions i ON i.contact_id = c.id WHERE c.user_id = 1
GROUP BY c.id HAVING last_contact IS NULL
OR last_contact < DATE_SUB(NOW(), INTERVAL 30 DAY) ORDER BY last_contact ASC;

SELECT
(SELECT COUNT(*) FROM contacts WHERE user_id = 1) AS total_contacts,
(SELECT COUNT(*) FROM interactions WHERE user_id = 1) AS total_interactions,
(SELECT COUNT(*) FROM reminders WHERE user_id=1 AND is_done=0 AND due_date>=NOW()) AS
pending_reminders,
```

```
(SELECT COUNT(*) FROM reminders WHERE user_id=1 AND is_done=0 AND due_date<NOW()) AS
overdue_reminders;

SELECT relationship_type, COUNT(*) AS total FROM contacts
WHERE user_id = 1 GROUP BY relationship_type ORDER BY total DESC;

SELECT category, COUNT(*) AS total FROM contacts WHERE user_id = 1 GROUP BY category;

SELECT interaction_type, COUNT(*) AS total FROM interactions
WHERE user_id = 1 GROUP BY interaction_type ORDER BY total DESC;

SELECT c.first_name, c.last_name, COUNT(i.id) AS interaction_count FROM contacts c
LEFT JOIN interactions i ON i.contact_id = c.id WHERE c.user_id = 1
GROUP BY c.id ORDER BY interaction_count DESC LIMIT 5;

SELECT MONTHNAME(interaction_date) AS month, COUNT(*) AS total FROM interactions
WHERE user_id = 1 AND YEAR(interaction_date) = YEAR(CURDATE())
GROUP BY MONTH(interaction_date) ORDER BY MONTH(interaction_date) ASC;
```

7. Login Window UI — login_window.py

```
import tkinter as tk
from tkinter import messagebox
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from database.db_connection import login_user, register_user, test_connection

BG_DARK="#0F1117"; BG_CARD="#1A1D2E"; BG_INPUT="#252840"
ACCENT="#6C63FF"; ACCENT_HOV="#8B84FF"; TEXT_PRI="#EAEAF5"
TEXT_SEC="#8888AA"; SUCCESS="#4ADE80"; ERROR="#F87171"; BORDER="#2E3150"
FONT_HEAD=("Segoe UI",22,"bold"); FONT_SUB=("Segoe UI",10)
FONT_LABEL=("Segoe UI",9); FONT_INPUT=("Segoe UI",11); FONT_BTN=("Segoe UI",10,"bold")

class LoginWindow:
def __init__(self, root: tk.Tk):
self.root = root
self.root.title("Relationship Tracker")
self.root.geometry("460x600")
self.root.resizable(False, False)
self.root.configure(bg=BG_DARK)
self._center_window()
self.mode = "login"
self.current_user = None
self._build_ui()

def _build_ui(self):
header = tk.Frame(self.root, bg=BG_DARK)
header.pack(fill="x", pady=(40, 0))
tk.Label(header, text="Relationship Tracker", font=FONT_HEAD, bg=BG_DARK, fg=TEXT_PRI).pack()
tk.Label(header, text="Personal & Professional connections, all in one place.",
font=FONT_SUB, bg=BG_DARK, fg=TEXT_SEC).pack(pady=(4, 0))
self.card = tk.Frame(self.root, bg=BG_CARD,
highlightbackground=BORDER, highlightthickness=1, bd=0)
self.card.pack(fill="both", padx=40, pady=30, ipady=20)
self.tab_frame = tk.Frame(self.card, bg=BG_CARD)
self.tab_frame.pack(fill="x", padx=20, pady=(20, 0))
self.tab_login = self._make_tab("Sign In", lambda: self._switch("login"))
self.tab_reg = self._make_tab("Register", lambda: self._switch("register"))
self.tab_login.pack(side="left", padx=(0, 4))
self.tab_reg.pack(side="left")
tk.Frame(self.card, bg=BORDER, height=1).pack(fill="x", padx=20)
self.fields_frame = tk.Frame(self.card, bg=BG_CARD)
self.fields_frame.pack(fill="both", padx=24, pady=16)
self.status_var = tk.StringVar()
self.status_lbl = tk.Label(self.card, textvariable=self.status_var,
```

```
font=FONT_LABEL, bg=BG_CARD, fg=ERROR, wraplength=360)
self.status_lbl.pack(padx=24, pady=(0, 4))
self.btn_submit = tk.Button(self.card, text="Sign In", font=FONT_BTN,
bg=ACCENT, fg="#FFFFFF", activebackground=ACCENT_HOV,
activeforeground="#FFFFFF", relief="flat", cursor="hand2",
bd=0, padx=12, pady=10, command=self._on_submit)
self.btn_submit.pack(fill="x", padx=24, pady=(0, 20))
self.btn_submit.bind("<Enter>", lambda e: self.btn_submit.config(bg=ACCENT_HOV))
self.btn_submit.bind("<Leave>", lambda e: self.btn_submit.config(bg=ACCENT))
self._switch("login")

def _make_tab(self, text, command):
return tk.Button(self.tab_frame, text=text, font=FONT_BTN,
bg=BG_CARD, fg=TEXT_SEC, activebackground=BG_CARD,
activeforeground=ACCENT, relief="flat", cursor="hand2",
bd=0, padx=16, pady=8, command=command)

def _switch(self, mode):
self.mode = mode; self.status_var.set("")
if mode == "login":
self.tab_login.config(fg=ACCENT, font=("Segoe UI",9,"bold"))
self.tab_reg.config(fg=TEXT_SEC, font=FONT_LABEL)
self.btn_submit.config(text="Sign In")
else:
self.tab_reg.config(fg=ACCENT, font=("Segoe UI",9,"bold"))
self.tab_login.config(fg=TEXT_SEC, font=FONT_LABEL)
self.btn_submit.config(text="Create Account")
for w in self.fields_frame.winfo_children(): w.destroy()
self._build_fields()

def _build_fields(self):
if self.mode == "register":
self.entry_email = self._field(self.fields_frame, "Email address", False)
self.entry_user = self._field(self.fields_frame, "Username", False)
self.entry_pass = self._field(self.fields_frame, "Password", True)
if self.mode == "register":
self.entry_pass2 = self._field(self.fields_frame, "Confirm password", True)
last = self.entry_pass2 if self.mode == "register" else self.entry_pass
last.bind("<Return>", lambda e: self._on_submit())

def _field(self, parent, placeholder, secret):
tk.Label(parent, text=placeholder, font=FONT_LABEL, bg=BG_CARD, fg=TEXT_SEC).pack(anchor="w",
pady=(8,2))
frame = tk.Frame(parent, bg=BG_INPUT, highlightbackground=BORDER, highlightthickness=1)
frame.pack(fill="x", pady=(0,2))
entry = tk.Entry(frame, font=FONT_INPUT, bg=BG_INPUT, fg=TEXT_PRI,
insertbackground=TEXT_PRI, relief="flat", bd=8, show="*" if secret else "")
```

```
        entry.pack(fill="x", ipady=4)
        entry.bind("<FocusIn>", lambda e, f=frame: f.config(highlightbackground=ACCENT))
        entry.bind("<FocusOut>", lambda e, f=frame: f.config(highlightbackground=BORDER))
        return entry

    def _on_submit(self):
        self.status_var.set("")
        if self.mode == "login": self._do_login()
        else: self._do_register()

    def _do_login(self):
        username = self.entry_user.get().strip(); password = self.entry_pass.get()
        if not username or not password:
            self._show_error("Please fill in all fields."); return
        self._set_loading(True); self.root.update()
        success, result = login_user(username, password)
        self._set_loading(False)
        if success:
            self.current_user = result
            self._show_success(f"Welcome back, {result['username']}!")
            self.root.after(900, self._open_dashboard)
        else:
            self._show_error(result)

    def _do_register(self):
        email=self.entry_email.get().strip(); username=self.entry_user.get().strip()
        password=self.entry_pass.get(); confirm=self.entry_pass2.get()
        if not all([email, username, password, confirm]):
            self._show_error("Please fill in all fields."); return
        if "@" not in email or "." not in email:
            self._show_error("Enter a valid email address."); return
        if len(username) < 3:
            self._show_error("Username must be at least 3 characters."); return
        if len(password) < 6:
            self._show_error("Password must be at least 6 characters."); return
        if password != confirm:
            self._show_error("Passwords do not match."); return
        self._set_loading(True); self.root.update()
        success, result = register_user(username, email, password)
        self._set_loading(False)
        if success:
            self._show_success("Account created! You can now sign in.")
            self.root.after(1200, lambda: self._switch("login"))
        else:
            self._show_error(result)

    def _show_error(self, msg): self.status_lbl.config(fg=ERROR); self.status_var.set(msg)
```

```
def _show_success(self, msg): self.status_lbl.config(fg=SUCCESS); self.status_var.set(msg)

def _set_loading(self, loading):

text = "Please wait..." if loading else ("Sign In" if self.mode=="login" else "Create Account")

self.btn_submit.config(text=text, state="disabled" if loading else "normal")

def _open_dashboard(self):

messagebox.showinfo("Login Successful",

f"Logged in as {self.current_user['username']}\nDashboard coming soon.")

def _center_window(self):

self.root.update_idletasks()

w, h = 460, 600

x = (self.root.winfo_screenwidth() - w) // 2

y = (self.root.winfo_screenheight() - h) // 2

self.root.geometry(f"{w}x{h}+{x}+{y}")

def main():

if not test_connection():

root = tk.Tk(); root.withdraw()

messagebox.showerror("Database Error",

"Could not connect to MySQL.\n1. Make sure MySQL is running.\n2. Check db_connection.py")

return

root = tk.Tk(); LoginWindow(root); root.mainloop()

if __name__ == "__main__":

main()
```

8. App Entry Point — main.py

```
from ui.login_window import main

if __name__ == "__main__":
main()
```

requirements.txt

```
mysql-connector-python>=8.3.0
bcrypt>=4.1.0
Pillow>=10.0.0
```

Demo Login: username = demo_user | password = demo1234
All queries use user_id = 1 by default. Replace with your actual user ID after login.